

Лекция 1

Основы программирования на языке C++

Литература

1. Шурыгин В.Н. Объектно-ориентированное программирование : конспект лекций. – Моск. гос. ун-т печати имени Ива-на Федорова. – М. : МГУП имени Ивана Федорова, 2014. – 164 с. – URL: http://storage.elib.mgup.ru/7/Shurigin_2014.pdf.
2. Лафоре Р. Объектно-ориентированное программирование в C++. – СПб. : ПИТЕР, 2004. – 922 с. – URL: https://codernet.ru/books/c_plus/obektno-orientirovannoe_programmirovanie_v_c/.
3. Страуструп Б. Язык программирования C++. Специальное издание. – М. : БИНОМ, 2011. – 1136 с. – URL: https://proklondike.net/books/cpp/strastrup_c_specz_2011.html.

1 Структура программы

```
// директивы

// объявление глобальных переменных
// объявление, определение функций

// главная функция main
int main()
{
    // тело функции main
}

// создание функции
Тип_возвращаемого_значения Имя_функции(
Список_параметров )
{
    // тело функции
}
```

Препроцессор – программа, подготавливающая код программы к компиляции. Работа препроцессора управляется **директивами**.

С помощью препроцессора можно выполнять следующие операции:

- ✓ включение в программу текстов из указанных файлов;
- ✓ определение пространства имен;
- ✓ макроподстановка;
- ✓ исключение из программы отдельных частей текста (условная компиляция).

```
1  #include <iostream> //подключение библиотеки iostream
2  #define god 19 //с этого места препроцессор будет заменять "god" на "19"
3  using namespace std; // использовать пространство имен std
4
5  int main ()
6  {
7      int a[god];
8
9      for (int i = 15; i < god; i++)
10     {
11         a[i] = i;
12         cout << i << endl;
13     }
14
15     #undef god //с этой точки препроцессор больше не будет
16             //заменять "god" на "19"
17
18     int god = 20;
19     cout << god << endl;
```

15
16
17
18
20

Определение пространства имен

std - пространство имён для функций, классов и объектов из стандартной библиотеки

```
using namespace std;
```

```
#include <iostream>  
using std::cin;  
using std::cout;  
  
#include <iomanip>  
using std::setw;
```

Макроподстановка

```
#define <идентификатор> <текст>  
#define <идентификатор>(<список параметров>) <текст>  
#undef <идентификатор>
```

```
1  #include <iostream>  
2  
3  #define SUM(val_1, val_2) val_1 + val_2  
4  
5  using namespace std;  
6  
7  int main ()  
8  {  
9      int a = 3, b = 2;  
10     cout << SUM (a,b);  
11 }
```

5

Исключение из программы отдельных частей текста (условная компиляция)

Директивы : `#if` , `#else` , `#elif` , `#endif` , ...

```
1  #if <условие>
2  ...
3  #endif
```

```
1  #if <условие>
2  ...
3  #else
4  ...
5  #endif
```

```
1  #if <условие>
2  ...
3  #elif <условие>
4  ...
5  #elif <условие>
6  ...
7  #else
8  ...
9  #endif
```

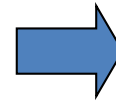
```
1  #include <iostream>
2  using namespace std;
3
4  # define flag 2
5
6  #if flag == 1
7      char mas[] = "one";
8  #elif flag == 2
9      char mas[] = "two";
10 #else
11     char mas[] = "three";
12 #endif
13
14 int main ()
15 {
16     cout << mas;
17 }
```

two

2 Базовые типы данных

1) Логический тип : **bool**

```
bool b1 = 100, b2 = 0;  
bool b3 = -100;  
bool b4 = true, b5 = false;  
  
cout << "b1 = " << b1 << endl << "b2 = " << b2 << endl;  
cout << "b3 = " << b3 << endl;  
cout << "b4 = " << b4 << "\nb5 = " << b5;
```



```
b1 = 1  
b2 = 0  
b3 = 1  
b4 = 1  
b5 = 0
```

2) Целочисленный тип : **int**

3) Вещественный тип : **float, double**

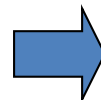
4) Пустой тип : **void**

5) Символьный тип :

char - один символ в кодировке ASCII

wchar_t - расширенный символ

```
char c1 = 'A';  
wchar_t c2 = 'A';  
  
cout << "c1 = " << c1 << endl;  
cout << "c2 = " << c2 << endl;  
wcout << "c2 = " << c2 << endl;
```



```
c1 = A  
c2 = 65  
c2 = A
```

Спецификаторы типов:

- 1) short;
- 2) long;
- 3) signed;
- 4) unsigned.

Пользовательские типы данных?

3 Преобразование типов

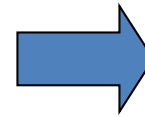
операция<новый тип>(выражение)

Знак операции	Наименование
static_cast	Преобразование с проверкой во время компиляции
dynamic_cast	Преобразование с проверкой во время выполнения (динамическое приведение типов). Имеет смысл только при использовании наследования и полиморфизма
reinterpret_cast	Преобразование без проверки
const_cast	Константное преобразование (выражение – ссылка или указатель)

```

15 double res1 = 13/7;
16 cout << res1 << endl;
17
18 double res2 = static_cast<double>(13)/7;
19 cout << res2 << endl;
20
21 double res3 = (double) 13/7;
22 cout << res3 << endl;

```



```

1
1.85714
1.85714

```

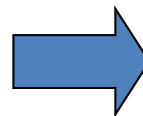
main.cpp: In function 'int main()':
 main.cpp:18:21: error: invalid static_cast from type 'int' to type 'int*'
 static_cast<int *>(a);
 ^



```

1  #include <iostream>
2  using namespace std;
3
4  int main ()
5  {
6  const int i = 11;
7  const int *ptr = &i;
8  cout << i << endl;
9  cout << *ptr << endl;
10
11  *(const_cast<int *>(ptr)) = 22;
12  cout << i << endl;
13  cout << *ptr;
14  }

```



```

15 int a;
16
17 reinterpret_cast<int *>(a);
18 static_cast<int *>(a);

```

```

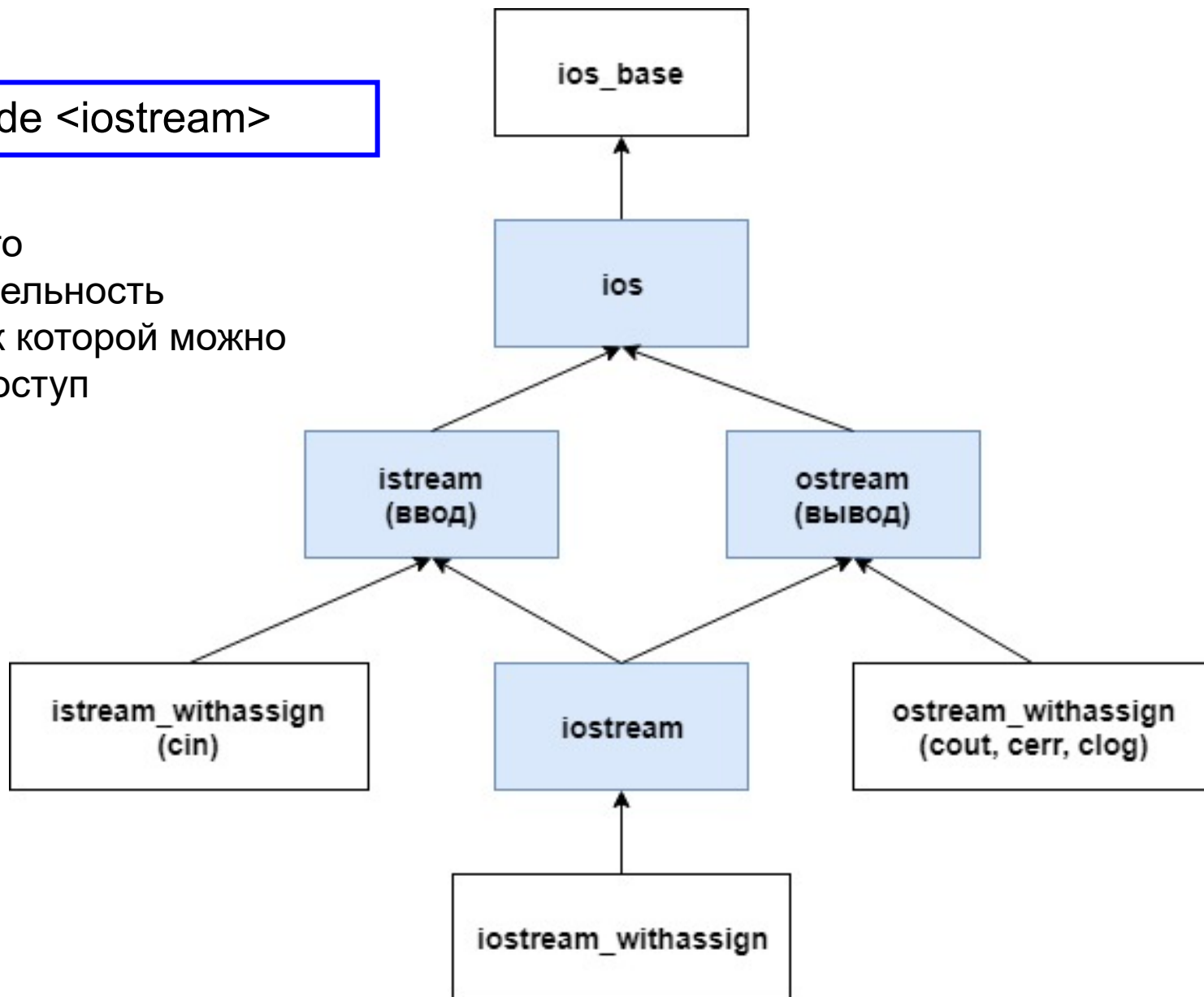
11
11
11
22

```

4 ПОТОКИ ВВОДА И ВЫВОДА

```
#include <iostream>
```

Поток — это
последовательность
символов, к которой можно
получить доступ



Поток ввода (или ещё «**входной поток**») используется для хранения данных, полученных от источника данных: клавиатуры, файла и т.д.

Класс istream используется для работы с входными потоками. **Оператор извлечения >>** используется для извлечения значений из потока.

Поток вывода (или ещё «**выходной поток**») используется для хранения данных, предоставляемых конкретному потребителю данных: монитору, файлу, принтеру и т.д.

Класс ostream используется для работы с выходными потоками. **Оператор вставки <<** используется для помещения значений в поток.

Класс istream может обрабатывать как ввод, так и вывод данных, что позволяет ему осуществлять двунаправленный ввод/вывод.

Класс **ios** определяет кучу разных вещей, которые являются общими для потоков ввода/вывода.

Стандартные объекты потоков:

- ✓ **cin** — класс `istream_withassign`, связанный со стандартным вводом (обычно это клавиатура);
- ✓ **cout** — класс `ostream_withassign`, связанный со стандартным выводом (обычно это монитор);
- ✓ **cerr** — класс `ostream_withassign`, связанный со стандартной ошибкой (обычно это монитор), обеспечивающий небуферизованный вывод;
- ✓ **clog** — класс `ostream_withassign`, связанный со стандартной ошибкой (обычно это монитор), обеспечивающий буферизованный вывод.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int x;
8     cout << "Введите целое число: ";
9     cin >> x;
10    cout << "Вы ввели число " << x;
11
12    return 0;
13 }
```



```
1 #include <iostream>
2
3 //using namespace std;
4
5 int main()
6 {
7     int x;
8     std::cout << "Введите целое число: ";
9     std::cin >> x;
10    std::cout << "Вы ввели число " << x;
11
12    return 0;
13 }
```



```
Введите целое число: 91
Вы ввели число 91
```

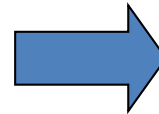
Функции неформатированного ввода/вывода

istream	ostream
<code>get()</code> - извлекает символ	<code>put()</code> - записывает символ
<code>peek()</code> - читает символ без извлечения	
<code>unget()</code> - возврат извлеченного ранее символа в поток	
<code>putback()</code> - вставка символа в поток	
<code>getline()</code> - извлекает символы, пока не будет найден заданный символ	
<code>ignore()</code> - удаляет из потока определенное количество символов пока не будет встречен символ разделитель, который также отбрасывается	
<code>read()</code> - извлекает из потока блоки символов в целевой массив	<code>write()</code> - вставляет блоки символов в поток
<code>gcount()</code> - количество извлеченных из потока символов. Все выше перечисленные функции будут воздействовать на <code>gcount</code> , включая <code>>></code> (операцию извлечения)	

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7  char ch = cin.peek();
8  cout << ch << endl;
9
10 char r = cin.get();
11 cout << r << endl;
12
13 char s[7];
14 cin.read(s, 7);
15 cout << s << endl;
16
17 char k = cin.get();
18 cout << k << endl;
19
20 cin.ignore(20, '\n');
21
22 char line = 'W';
23 cout.put(line);
24
25 char str1[] = "p r o g r a m m i n g";
26 cout.write(str1, 5);
27 cout << endl;
28
29 char s1[10];
30 cin >> s1;
31 cout << s1;
32 }

```



```

A
A
A
Hel lo,world!

Hel lo
,
Wp r o
Hello, world!
Hello,

```

5 Манипуляторы ввода вывода

Флаги форматирования (1)

Флаг	Назначение	Пример	Результат
boolalpha	Вывод логических величин в текстовом виде (true , false)	<pre>cout.setf(ios::boolalpha); bool log_false = 0, log_true = 1; cout << log_false << endl << log_true << endl;</pre>	false true
oct	Ввод/вывод величин в восьмеричной системе счисления (сначала снимаем флаг dec , затем устанавливаем флаг oct)	<pre>cout.unsetf(ios::dec); cout.setf(ios::oct); int value; cin >> value; cout << value << endl;</pre>	ВВОД: 99 ₁₀ ВЫВОД: 143 ₈
dec	Ввод/вывод величин в десятичной системе счисления (флаг установлен по умолчанию)	<pre>cout.setf(ios::dec); int value = 148; cout << value << endl;</pre>	148
hex	Ввод/вывод величин в шестнадцатеричной системе счисления (сначала снимаем флаг dec , затем устанавливаем флаг hex)	<pre>cout.unsetf(ios::dec); cout.setf(ios::hex); int value; cin >> value; cout << value << endl;</pre>	ВВОД: 99 ₁₀ ВЫВОД: 63 ₁₆

Флаги форматирования (2)

showbase	Выводить индикатор основания системы счисления	<pre>cout.unsetf(ios::dec); cout.setf(ios::oct ios::showbase); int value; cin >> value; cout << value << endl;</pre>	ВВОД: 99_{10} ВЫВОД: 0143_8
uppercase	В шестнадцатеричной системе счисления использовать буквы верхнего регистра(по умолчанию установлены буквы нижнего регистра)	<pre>cout.unsetf(ios::dec); cout.setf(ios::hex ios::uppercase); int value; cin >> value; cout << value << endl;</pre>	ВВОД: 255_{10} ВЫВОД: FF_{16}
showpos	Вывод знака плюс (+) для положительных чисел	<pre>cout.setf(ios::showpos); int value = 15; cout << value << endl;</pre>	+15
scientific	Вывод чисел с плавающей точкой в экспоненциальной форме	<pre>cout.setf(ios::scientific); double value = 1024.165; cout << value << endl;</pre>	1.024165e+003

Манипуляторы форматирования (1)

Манипулятор	Назначение	Пример	Результат
endl	Переход на новую строку при выводе	<code>cout << «website:» << endl << «cppstudio.com»;</code>	website: cppstudio.co m
boolalpha	Вывод логических величин в текстовом виде (true , false)	<code>bool log_true = 1; cout << boolalpha << log_true << endl;</code>	true
nboolalpha	Вывод логических величин в числовом виде (true , false)	<code>bool log_true = true; cout << nboolalpha << log_true << endl;</code>	1
oct	Вывод величин в восьмеричной системе счисления	<code>int value = 64; cout << oct << value << endl;</code>	100₈
dec	Вывод величин в десятичной системе счисления (по умолчанию)	<code>int value = 64; cout << dec << value << endl;</code>	64₁₀
hex	Вывод величин в шестнадцатеричной системе счисления	<code>int value = 64; cout << hex << value << endl;</code>	40₈
showbase	Выводить индикатор основания системы счисления	<code>int value = 64; cout << showbase << hex << value << endl;</code>	0x40

Манипуляторы форматирования (2)

noshowbase	Не выводить индикатор основания системы счисления (по умолчанию).	int value = 64; cout << noshowbase << hex << value << endl;	40
uppercase	В шестнадцатеричной системе счисления использовать буквы верхнего регистра (по умолчанию установлены буквы нижнего регистра).	int value = 255; cout << uppercase << hex << value << endl;	FF₁₆
nouppercase	В шестнадцатеричной системе счисления использовать буквы нижнего регистра (по умолчанию).	int value = 255; cout << nouppercase << hex << value << endl;	ff₁₆
showpos	Вывод знака плюс (+) для положительных чисел	int value = 255; cout << showpos << value << endl;	+255
noshowpos	Не выводить знак плюс (+) для положительных чисел (по умолчанию).	int value = 255; cout << noshowpos << value << endl;	255
scientific	Вывод чисел с плавающей точкой в экспоненциальной форме	double value = 1024.165; cout << scientific << value << endl;	1.024165e +003
fixed	Вывод чисел с плавающей точкой в фиксированной форме (по умолчанию).	double value = 1024.165; cout << fixed << value << endl;	1024.165

Манипуляторы форматирования (3)

setw(int number)	Установить ширину поля, где number — количество позиций, символов (выравнивание по умолчанию по правой границе). Манипулятор с параметром.	<code>cout << setw(40) << «cppstudio.com» << endl;</code>	<code>__cppstudio.com</code>
right	Выравнивание по правой границе (по умолчанию). Сначала необходимо установить ширину поля (ширина поля должна быть заведомо большей чем, длина выводимой строки).	<code>cout << setw(40) << right << «cppstudio.com» << endl;</code>	<code>__cppstudio.com</code>
left	Выравнивание по левой границе. Сначала необходимо установить ширину поля (ширина поля должна быть заведомо большей чем, длина выводимой строки).	<code>cout << setw(40) << left << «cppstudio.com» << endl;</code>	<code>cppstudio.com__</code>
setprecision(int count)	Задаёт количество знаков после запятой, где count — количество знаков после десятичной точки	<code>cout << fixed << setprecision(3) << (13.5 / 2) << endl;</code>	<code>6.750</code>
setfill(int symbol)	Установить символ заполнитель. Если ширина поля больше, чем выводимая величина, то свободные места поля будут наполняться символом symbol — символ заполнитель	<code>cout << setfill('0') << setw(4) << 15 << ends << endl;</code>	<code>0015</code>

6 Циклы и ветвления

for, while, do-while

if, switch

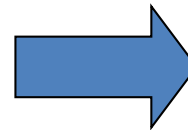
if (условие) оператор; **else** оператор;

7 Функции

Способы передачи аргументов в функцию:

- ✓ передача аргумента по значению;
- ✓ передача с использованием указателя на аргумент;
- ✓ передача аргумента по ссылке.

```
1  #include <iostream>
2  using namespace std;
3
4  void SUM (int a2, int b2, int &c2)
5  {
6      c2 = a2+b2;
7      a2++;
8      b2++;
9  }
10
11 int main()
12 {
13     int a1 = 2, b1 = 3, c1 = 0;
14     SUM(a1,b1,c1);
15     cout << "a1 = " << a1 << endl;
16     cout << "b1 = " << b1 << endl;
17     cout << "c1 = " << c1;
18 }
```



```
a1 = 2
b1 = 3
c1 = 5
```

```

1  #include <iostream>
2  using namespace std;
3
4  void pass_by_value (int a)
5  {
6  a ++; cout<<"pass_by_value : a = " << a << endl;
7  }
8
9  void pass_by_pointer (int* a)
10 {
11 *a +=2; cout << "pass_by_pointer : a = " << *a << endl;
12 }
13
14 void pass_by_reference (int& a)
15 {
16 a +=3; cout << "pass_by_reference : a = " << a << endl;
17 }
18
19 int main()
20 {
21 int a;
22 cout << "a = "; cin >> a;
23
24 pass_by_value (a);
25 cout << "main : a = " << a << endl;
26
27 pass_by_pointer (&a);
28 cout << "main : a = " << a << endl;
29
30 pass_by_reference (a);
31 cout << "main : a = " << a << endl;
32 return 0;
33 }

```

```

a = 5
pass_by_value : a = 6
main : a = 5
pass_by_pointer : a = 7
main : a = 7
pass_by_reference : a = 10
main : a = 10

```

Перегруженные функции — это функции с одним именем и одного типа, но с разными типами аргументов и их количеством

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int summa(int a, int b)
6  {
7      cout << "Складываем два целых числа "; return a+b;
8  }
9
10 int summa(int a, int b, int c)
11 {
12     cout << "Складываем три целых числа "; return a+b+c;
13 }
14
15 int summa(float a, float b)
16 {
17     cout << "Складываем два дробных числа ";
18     return floor(a+b); //выделение целой части
19 }
20
21 int main()
22 {
23     int a, b, c; float d, e;
24     cout << "Введите два целых числа ";
25     cin >> a >> b;
26     cout << a << " + " << b << " = " << summa(a,b) << endl;
27     cout << "Введите еще одно целое число ";
28     cin >> c;
29     cout << a << " + " << b << " + " << c << " = " << summa(a,b,c) << endl;
30     cout << "Введите два дробных числа ";
31     cin >> d >> e;
32     cout << d << " + " << e << " = " << summa(d,e) << endl;
33 }
```

```
Введите два целых числа 2 3
Складываем два целых числа 2 + 3 = 5
Введите еще одно целое число 4
Складываем три целых числа 2 + 3 + 4 = 9
Введите два дробных числа 1.22 3.44
Складываем два дробных числа 1.22 + 3.44 = 4
```

Встраиваемые функции (inline) и аргументы по умолчанию

```
1  #include <iostream>
2  using namespace std;
3
4  inline float sum(float x = 0.0F, float y = 0.0F)
5  {
6  return (x * x + y * y);
7  }
8
9  int main()
10 {
11 float a, b;
12 cout << "Введите два числа ";
13 cin >> a >> b;
14 cout << "a^2 + b^2 = " << sum(a,b) << endl;
15
16 cout << "a^2 = " << sum(a) << endl; /*верно,
17 т.к. по умолчанию второй аргумент равен нулю.*/
18
19 cout << "sum = " << sum() << endl;
20 return 0;
21 }
```

```
Введите два числа 2 3
a^2 + b^2 = 13
a^2 = 4
sum = 0
```

Константные аргументы функции

Передаче аргумента по значению - замедление программы из-за копирования.

Константный аргумент функции - передача аргумента по ссылке с запретом его изменения.

```
1  #include <iostream>
2  using namespace std;
3
4  inline float sum(const float& x, const float& y = 0.0F)
5  {
6  return (x * x + y * y);
7  }
8
9  int main()
10 {
11 float a, b;
12 cout << "Введите два числа ";
13 cin >> a >> b;
14 cout << "a^2 + b^2 = " << sum(a,b) << endl;
15
16 cout << "a^2 = " << sum(a) << endl; /*верно,
17 т.к. по умолчанию второй аргумент равен нулю.*/
18 return 0;
19 }
```

```
Введите два числа 2 5
a^2 + b^2 = 29
a^2 = 4
```